

Informe proyecto RAG médico junto a sistema de memoria, planificación.

Integrantes: Ignacio Mella Cordero, Benjamin Mella Cordero.

Sección: 002D

Fecha: 27/10/2025

El Backend se encuentra en el archivo `agente_medico_mejorado.py` donde se define la clase `MedicalAgentImproved`, la clase encapsula toda la lógica del procesamiento de lenguaje, RAG, gestión de memoria y ejecución de herramientas especializadas.

El Frontend se encuentra en el archivo `app_medico.py` donde se construye una app web interactiva construida con `streamlit`, actúa como el cliente que consume la instancia backend, gestiona el estado de la sesión del usuario y renderiza la interfaz.

Componentes Backend:

inicialización:

- Se genera la configuración LLM, inicializando `ChatOpenAI` apuntando a la API de github models que se debe configurar en el archivo `.env`
- Contiene un sistema RAG donde se deben cargar documentos del tipo pdf, procesa y divide los documentos y crea e indexa un Chroma con embeddings.
- Contiene una instancia `ConversationSummaryMemory` para optimizar los tokens en conversaciones largas, o puede utilizar `ConversationBufferMemory` para un historial completo.
- Contiene un conjunto de Tool de LangChain que encapsula tanto la lógica de RAG y análisis.
- Utiliza `create_openai_functions_agent` con un prompt de sistema que instruye al agente sobre planificación jerárquica y triaje de seguridad, además envuelve el agente en un `AgentExecutor` el cual gestiona la memoria y la ejecución de herramientas.

Herramientas:

- Las herramientas `BuscarSintomas`, `BuscarTratamientos` y `BuscarMedicamentos` consultan el vectorstore de Chroma.
- Las herramientas `AnalisisDiferencial` y `ValidarInteracciones` usan RAG para obtener contexto y luego realizan una llamada LLM estructurada para un análisis complejo.

- DetectarEmergencia realiza un triaje inmediato donde compara la entrada con una lista CRITICAL_SYMPTONS predefinida.
- EvaluarRelevancia utiliza cosine_similarity_native para poder puntuar la relevancia que hay entre la consulta y el contexto recuperado.

Interfaz de ejecución:

- Es el punto de entrada principal para el frontend, invoca self.agent_executor.invoke().
- Calcula el tiempo de respuesta.
- Ejecuta _evaluate_response_quality para poder puntuar la respuesta.
- Retorna un diccionario estructurado con la respuesta y metadatos.

Funciones de Gestión:

- Get_statistics() retorna métricas de la sesión.
- Get_memory_summary() extrae el estado actual de la memoria.
- Reset_memory() limpia el historial de la memoria del agente.

Componente Frontend:

Gestión de estado:

- St.session_state.agent almacena la instancia única de la clase MedicalAgentImproved después de la inicialización.
- St.session_state.chat_history contiene la lista de diccionarios que almacena la conversación para lograr ser renderizada.
- St.session_state.quality_scores contiene la lista de puntuaciones para el grafico de estadísticas.

Flujo de interfaz:

- Configuración: El usuario define la ruta pdf_directory, el modelo LLM y el tipo de memoria en la barra lateral.
- Inicialización: El usuario presiona “Inicializar agente”, donde se ejecuta la función initialize_agent(), la cual instancia MedicalAgentImproved con la configuración de la barra lateral y la guarda en st.session_state.agent
- Interacción: El usuario ingresa una consulta en st.text_area y presiona el botón “Enviar”, luego de eso se ejecuta la función process_question(question)
- Procesamiento: process_question añade el mensaje del usuario y llama al método consult() del backend.

- Recibe el diccionario de respuesta del agente y añade la respuesta del asistente a `st.session_state.chat_history` y actualiza las métricas.
- Renderizado: La app se actualiza y la función `render_chat_message` dibuja un nuevo historial de chat
- Estadísticas: La pestaña “Estadísticas” llama a `st.session_state.agent.get_statistics()` y `get_memory_summary()` para mostrar las métricas en tiempo real.

Diagrama de arquitectura:

